

10 Exercícios de Orientação a Objetos

Foco: encapsulamento, composição, classes estáticas, wrapper classes, String e API Date/Time. Dificuldade crescente. Implemente em Java.

1. Encapsulamento básico — ContaBancaria

Crie a classe ContaBancaria com os atributos privados numero (String), titular (String) e saldo (double). Implemente:

- Construtor que recebe numero e titular, iniciando saldo em 0.
- Método depositar(double valor), que só aceita valores positivos.
- Método sacar(double valor), que só permite saque se houver saldo suficiente, retornando true/false conforme o sucesso.
- Getters para todos os atributos, sem setter para saldo (o saldo só muda via depósito/saque).

Objetivo: praticar por que o saldo não deve ter um setter público — a lógica de alteração deve ficar dentro da própria classe.

2. Composição simples — Pessoa e Endereco

Crie a classe Endereco (rua, numero, cidade, cep) e a classe Pessoa (nome, cpf e um objeto Endereco). Uma Pessoa deve ser criada já com seu Endereco (associação de composição — o Endereco não existe sem a Pessoa). Implemente um método exibirDados() em Pessoa que também mostra os dados do endereço, delegando a formatação para um método toString() ou similar dentro da própria classe Endereco.

3. Wrapper classes — autoboxing, unboxing e comparação

Crie uma classe Estatistica com um método media(Integer[] valores) que calcula a média dos valores (cuidado com unboxing de valores null — devem ser ignorados no cálculo). Depois, escreva um pequeno teste que demonstre e comente, em Java (via comentários no código), a diferença entre comparar dois objetos Integer com == e com .equals(), mostrando um caso em que == falha (dica: valores fora do cache de Integer, acima de 127).

4. String — manipulação de texto

Crie a classe Formatador com métodos estáticos:

- capitalizar(String texto): deixa a primeira letra de cada palavra maiúscula.
- somenteNumeros(String texto): remove todo caractere que não seja dígito (use métodos de String, sem regex nesta questão).
- truncar(String texto, int tamanho): corta o texto no tamanho informado, adicionando "..." se ele foi cortado.

Use os métodos da API String (substring, toUpperCase, toCharArray, trim, etc.) — lembrando que String é imutável em Java.

5. Composição com array de objetos — Turma e Aluno

Crie a classe Aluno (nome, matricula, notas: double[]) com um método media() que calcula a média das notas. Crie a classe Turma, que possui um array de Aluno (composição: os Alunos existem dentro do contexto da Turma).

Implemente em Turma:

- adicionarAluno(Aluno a), respeitando o tamanho máximo do array.
- melhorAluno(), que retorna o Aluno com maior média.
- mediaGeral(), que retorna a média de todas as médias da turma.

6. Classes estáticas — contador de instâncias e utilitário

Crie a classe Produto com um atributo estático contador, incrementado a cada novo objeto criado no construtor, e um método estático totalDeProdutos(). Em seguida, crie uma classe utilitária ConversorMoeda (construtor privado, apenas métodos estáticos) com métodos estáticos reaisParaDolar(double valor, double cotacao) e dolarParaReais(double valor, double cotacao). Justifique (em comentário) por que essa classe não deveria ser instanciada.

7. API Date/Time — calcular idade

Crie a classe Pessoa (nome e LocalDate dataNascimento). Implemente o método int idade(), que calcula a idade atual da pessoa em anos completos usando o pacote java.time (dica: java.time.Period). Implemente também aniversariouEsteAno(): retorna true se a data de aniversário já passou no ano corrente.

8. API Date/Time — duração entre eventos

Crie a classe Tarefa (descricao, LocalDateTime inicio, LocalDateTime fim — fim pode ser null se a tarefa ainda está em andamento). Implemente:

- Duration duracao(): calcula a duração entre início e fim (lance uma exceção, ou retorne null, se fim ainda não foi definido).
- String duracaoFormatada(): retorna a duração no formato "Xh Ymin".
- boolean estaAtrasada(LocalDateTime prazo): compara se fim (ou o momento atual, se ainda não terminou) está após o prazo informado.

Use os métodos apropriados de LocalDateTime e Duration (toHours, toMinutes, isBefore, isAfter, etc.).

9. String + regex básico — validação de dados

Crie a classe Validador com métodos estáticos que usam expressões regulares simples (java.util.regex ou String.matches):

- boolean emailValido(String email): valida um formato básico do tipo texto@texto.dominio.
- boolean somenteLetras(String texto): valida se contém apenas letras (com ou sem espaços).
- String[] separarNomeSobrenome(String nomeCompleto): usa split para separar o primeiro nome do restante.

Não é necessário regex avançado — o objetivo é praticar os padrões mais comuns (\\d, \\w, +, *, ^, \$).

10. Integração — Sistema de Biblioteca (composição + String + Date/Time)

Este é o exercício mais completo, unindo tudo que foi praticado. Modele:

- Livro: titulo, codigo (deve seguir o padrão "LIV-000" — três dígitos, validado com regex), disponivel (boolean).
- Usuario: nome, matricula.
- Emprestimo: composição de um Livro e um Usuario, LocalDate dataEmprestimo e LocalDate dataDevolucaoPrevista (7 dias após o empréstimo).
- Biblioteca: mantém um array de Emprestimo (composição), com métodos realizarEmprestimo(Livro, Usuario) — que marca o livro como indisponível — e devolver(Emprestimo), que calcula, via Period ou o cálculo de dias entre datas, se houve atraso e quantos dias, cobrando uma multa fictícia de R\$1,00 por dia de atraso.

Capriche no encapsulamento: nenhum atributo deve ser alterado diretamente de fora das classes; toda alteração de estado (livro disponível/indisponível, multa) deve acontecer através de métodos.